Tokens, Keywords, Identifiers, Variables and Constants

Theory and Explanation — C Programming Language

1. Tokens

A **token** is the smallest individual unit in a C program that is meaningful to the compiler. When the compiler reads a source file, the first phase (called **lexical analysis**) breaks the raw text into a stream of tokens. The compiler cannot process a program as one long string of text — it must first split it into these smallest meaningful pieces.

For example, consider the statement below:

```
int age = 25;
```

This single line is broken into **5 tokens**: `int`, `age`, `=`, `25`, `;` — each recognized separately by the compiler.

Classification of Tokens in C

Token Type	Description	Examples
Keywords	Reserved words with predefined meaning	int, if, for, return
Identifiers	Names given by the programmer	age, total_sum, main
Constants	Fixed values that do not change	10, 3.14, 'A', "Hi"
Strings	Sequence of characters in double quotes	"Hello World"
Operators	Symbols that perform operations	+, -, *, /, ==, &&
Special Symbols	Punctuation used for syntax	{ } ( ) ; , #

2. Keywords

Keywords (also called reserved words) are predefined words in C that already have a fixed meaning known to the compiler. Because their meaning is built into the language, keywords:

- Cannot be used as identifiers (variable names, function names, etc.)
- Are always written in lowercase in standard C
- Cannot be redefined or redeclared

The C language (ANSI C) defines **32 keywords**. Some common ones are shown below:

`auto`	`break`	`case`	`char`	`const`
`continue`	`default`	`do`	`double`	`else`
`enum`	`extern`	`float`	`for`	`goto`
`if`	`int`	`long`	`register`	`return`
`short`	`signed`	`sizeof`	`static`	`struct`
`switch`	`typedef`	`union`	`unsigned`	`void`

`volatile`	`while`			

Example: In the code `if (a > b) return a;`, the words `if` and `return` are keywords — the compiler interprets them exactly as the language defines, never as variable names.

3. Identifiers

An **identifier** is the name given by the programmer to a program element such as a variable, function, array, structure, or label. Identifiers are how we refer to data and code throughout a program.

Rules for Naming Identifiers in C

- Must begin with a letter (A–Z, a–z) or an underscore (_)
- Can be followed by letters, digits (0-9), or underscores
- Cannot contain spaces or special symbols (@, #, %, -, etc.)
- Cannot be a C keyword (e.g. int, for, while)
- Are case-sensitive — `Sum` and `sum` are different identifiers
- Should not exceed 31 characters (implementation dependent, but this is safe practice)

Valid Identifiers	Invalid Identifiers	Reason (invalid)
`total_sum`	`2total`	Cannot start with a digit
`_count`	`total-sum`	Hyphen not allowed
`studentName`	`float`	float is a reserved keyword
`marks1`	`my name`	Space not allowed

4. Variables

A **variable** is a named location in memory that is used to store a value which **can change** during the execution of a program. Every variable in C has a **name** (identifier), a **data type**, and a **value** stored in memory.

Declaring and Initializing a Variable

```
int age; // Declaration: reserves memory, no value yet age = 20; //
Assignment: stores a value int marks = 90; // Declaration + Initialization
together marks = 95; // Value changed — this is why it is called 'variable'
```

Types of Variables (based on scope/lifetime)

Type	Description
Local Variable	Declared inside a function/block; accessible only within it

Global Variable	Declared outside all functions; accessible throughout the program
Static Variable	Retains its value between function calls
Register Variable	Suggests storage in a CPU register for faster access

5. Constants

A **constant** is a value that **cannot be changed** during the execution of a program. Unlike variables, once a constant is defined, its value remains fixed throughout the program.

Types of Constants in C

Type	Example	Description
`Integer Constant`	`10, -25, 0`	Whole numbers, positive or negative
`Floating-point Constant`	`3.14, -0.5`	Numbers with a decimal point
`Character Constant`	`'A', '9', '$'`	A single character in single quotes
`String Constant`	`"Hello"`	Sequence of characters in double quotes
`Enumeration Constant`	`enum {RED, GREEN}`	Named integer constants

Ways to Define Constants in C

```
#define PI 3.14159 // Using the preprocessor directive const float PI2 = 3.14159; // Using the 'const' keyword
```

With `#define`, the preprocessor replaces every occurrence of the name with its value before compilation. With `const`, the compiler enforces that the variable's value is never modified after initialization — attempting to change it produces a compile-time error.

6. Summary — Variables vs. Constants

Aspect	Variable	Constant
Value	Can change during execution	Fixed, cannot change
Declaration	int x = 5;	const int x = 5; / #define X 5
Purpose	Store data that varies	Store data that stays fixed
Memory	Allocated, value updatable	Allocated (or substituted by preprocessor)

In short: **Tokens** are the smallest units the compiler recognizes; **Keywords** are reserved tokens with fixed meaning; **Identifiers** are programmer-defined names; **Variables** are named memory locations whose values can change; and **Constants** are fixed values that never change during execution.